CDS 6334

Visual Information Processing

Assignment

Image Enhancement, filtering and edge detection using classical image processing techniques

Student ID	Student Name
243UC247P5	Muhammad Fitri Bin Yusa’

Table of Contents

1.0 Introduction

1.1 Background

Image processing plays a vital role in many current applications, such as medical imaging, surveillance systems, driverless cars, and industrial inspection. In most practical situations, photographs are not taken under ideal conditions. Their quality and usefulness may be reduced by bad lighting, low contrast or unwanted noise. Classical image processing techniques offer a structured approach to enhance image quality without reliance on machine learning or deep learning methods. In this assignment, we have used three basic techniques: Image Enhancement, Image Filtering, and Edge Detection. Each of these approaches addresses a certain difficulty in the image processing pipeline.

1.2 Problem Statement

The raw images captured by a camera or sensor may sometimes contain visual noise that makes them hard to interpret. Images with low contrast may hide critical details, images with noise may introduce unnecessary changes, while images with undefined object boundaries will not allow easier detection of those objects. This means that there will be limitations when analysing the images. Therefore, it becomes important to have an organized image processing procedure.

1.3 Objectives

The objective of this assignment is as follows:

- To apply and understand image enhancement techniques
- To apply image filtering techniques for noise reduction and image smoothing
- To apply edge detection techniques to detect object boundaries
- To analyse and interpret the results using proper discussion
- To present the result in a well-structured technical report

2.0 Methodology

2.1 Dataset

The dataset provided contains collections of photos with various properties, such as low contrast, noise, natural scenes, and object images. The second set of data is specifically for processing using edge detection techniques. The photos were loaded and converted to grayscale, which are single-channel pixel values ranging from 0 to 255, making it easier to apply standard image processing techniques.

2.2 Image Enhancement

Image enhancement provides the initial phase of the processing pipeline. The objective is to enhance the image's visual quality, thereby increasing the visibility of details and refining subsequent processing steps, including filtering and edge detection, to achieve higher-quality results. In this project, we will apply two enhancement techniques: Histogram Equalization and Gamma Correction.

2.2.1 Histogram Equalization

Histogram Equalization is a technique which involves changing pixel values to achieve a better visual contrast from 0 to 255. The histogram represents how often each value of pixel appears on an image. For a low-contrast image, it is clear that the histogram is normally found in a smaller range of the graph because it shows the same brightness among most of the pixels. Histogram equalization works on finding the cumulative density function (CDF) and applying it on each pixel so as to create a uniform distribution of pixel values over the full range.

```python
def compute_histogram(image):
    """Count frequency of each pixel value (0-255)"""
    hist = np.zeros(256, dtype=int)
    for val in image.flatten():
        hist[val] += 1
    return hist

def histogram_equalization(image):
    """Spread pixel values evenly across 0-255"""
    hist = compute_histogram(image)

    # Running total of histogram
    cdf = np.cumsum(hist)

    # Normalize to 0-255 range
    h, w = image.shape
    cdf_min = cdf[cdf > 0].min()

    # Build lookup table — maps old pixel value to new one
    lut = np.round(
        (cdf - cdf_min) / ((h * w) - cdf_min) * 255
    ).astype(np.uint8)

    # Apply lookup table to every pixel
    return lut[image]
```

✓ 0.0s

Figure 1: Histogram Equalization code snippet

2.2.2 Gamma Correction

The gamma correction process uses a power function to alter the overall brightness of the image. First of all, the pixel values are scaled to lie between 0.0 and 1.0. Next, they are raised to the power of 1/gamma and then scaled to the values of 0 to 255. When the gamma value is greater than 1, the image gets brighter with lower brightness levels getting enhanced more than those at higher levels. Conversely, when the gamma value is less than 1, the image gets darker.

```python
def gamma_correction(image, gamma=1.5):
    #gamma < 1 → brightens dark images
    #gamma > 1 → darkens bright/washed-out images
    # Normalize to 0-1, apply gamma, scale back to 0-255

    normalized = image / 255.0
    corrected = np.power(normalized, 1.0 / gamma)
    return (corrected * 255).astype(np.uint8)
```

✓ 0.0s

Figure 2: Gamma Correction code snippet

2.3 Image Filtering

Image filtering is the second step in this process, preceded by enhancement. The purpose of this stage is to reduce noise in the image while retaining only the important structures. This can be done by convolution, whereby a small matrix of numbers, called a kernel, moves over each pixel in the image, multiplying its elements by the surrounding pixel elements and summing them to generate a new pixel value. In our project, we shall employ two image filters – the Gaussian filter and the median filter.

2.3.1 Gaussian Filter

The Gaussian filter works by substituting each pixel with an average of other surrounding pixels according to a formula based on a Gaussian distribution. Pixels close to the central point of the filter kernel are accorded greater weight, resulting in a natural smoothing effect that reduces the appearance of noise and jagged lines. However, the Gaussian filter is prone to blurring due to the averaging method it employs.

```
#Gaussian kernel function: create a 2D Gaussian kernel of given size and sigma
def gaussian_kernel(size=5, sigma=1.0):
    k = size // 2
    x, y = np.mgrid[-k:k+1, -k:k+1]
    kernel = np.exp(-(x**2 + y**2) / (2 * sigma**2))
    return kernel / kernel.sum()

# gaussian filter
def gaussian_filter(image, size=5, sigma=1.0):
    kernel = gaussian_kernel(size, sigma)
    return convolve2d(image, kernel)
```

Figure 3: Gaussian kernel code snippet

2.3.2 Median Filter

On the other hand, the median filter operates by substituting each pixel value with the median value obtained from neighboring pixels. As opposed to the Gaussian filter, the median filter uses an actual pixel value rather than computing an average. This property makes the median filter extremely effective in dealing with salt and pepper noise, which refers to the occurrence of random black and white specks in the picture.

```python
# convolution function: slide kernel across image and compute weighted average
def convolve2d(image, kernel):
    kh, kw = kernel.shape
    pad_h, pad_w = kh // 2, kw // 2

    # Pad edges so border pixels get processed too
    padded = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='reflect')

    output = np.zeros_like(image, dtype=np.float64)
    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            region = padded[i:i+kh, j:j+kw]
            output[i, j] = np.sum(region * kernel)

    return np.clip(output, 0, 255).astype(np.uint8)

# meidan filter
def median_filter(image, size=3):
    pad = size // 2
    padded = np.pad(image, pad, mode='reflect')
    output = np.zeros_like(image)

    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            region = padded[i:i+size, j:j+size]
            output[i, j] = np.median(region)

    return output.astype(np.uint8)
```

Figure 4: Convolution filter (median) code snippet

2.4 Edge Detection

The final process step in this workflow is edge detection. The existence of edges in an image depicts that there is a sudden change in pixel intensity that usually refers to objects or surfaces. Edge detection plays a critical role in areas like object detection, shape identification, and scene analysis. Canny's edge detection technique has been chosen in this experiment.

2.4.1 Canny Edge Detector

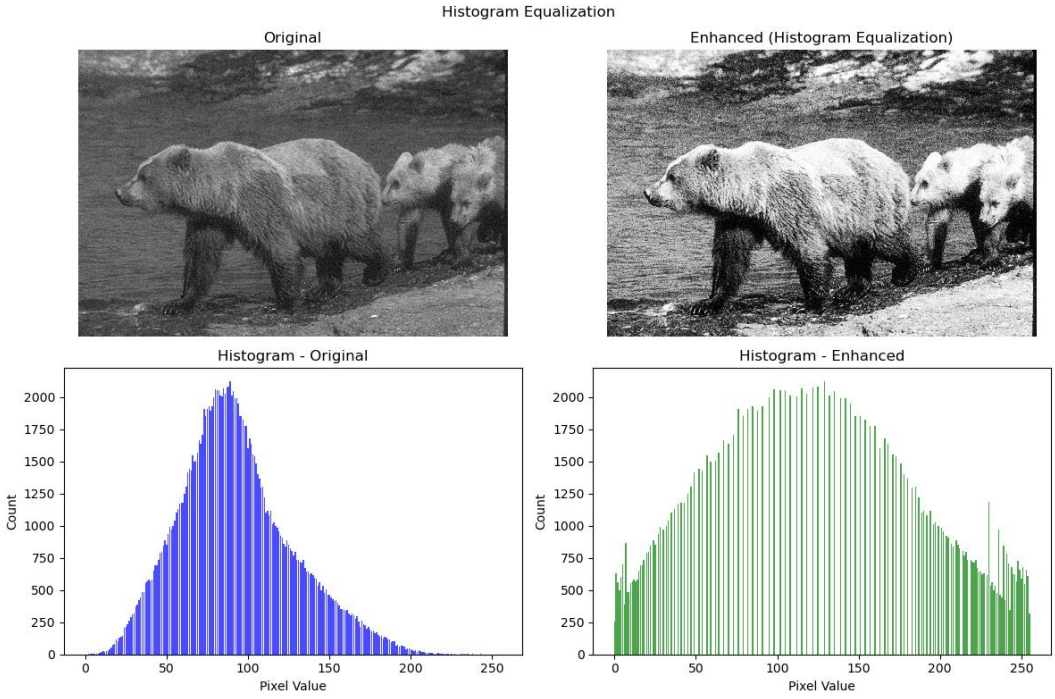
There are five main steps involved in Canny's edge detection algorithm. The first step involves applying a Gaussian filter to reduce noise in the image. This noise may be wrongly considered to be an edge. Secondly, Sobel kernels are used in both horizontal and vertical directions to determine the intensity and direction of the gradient at any particular pixel point.

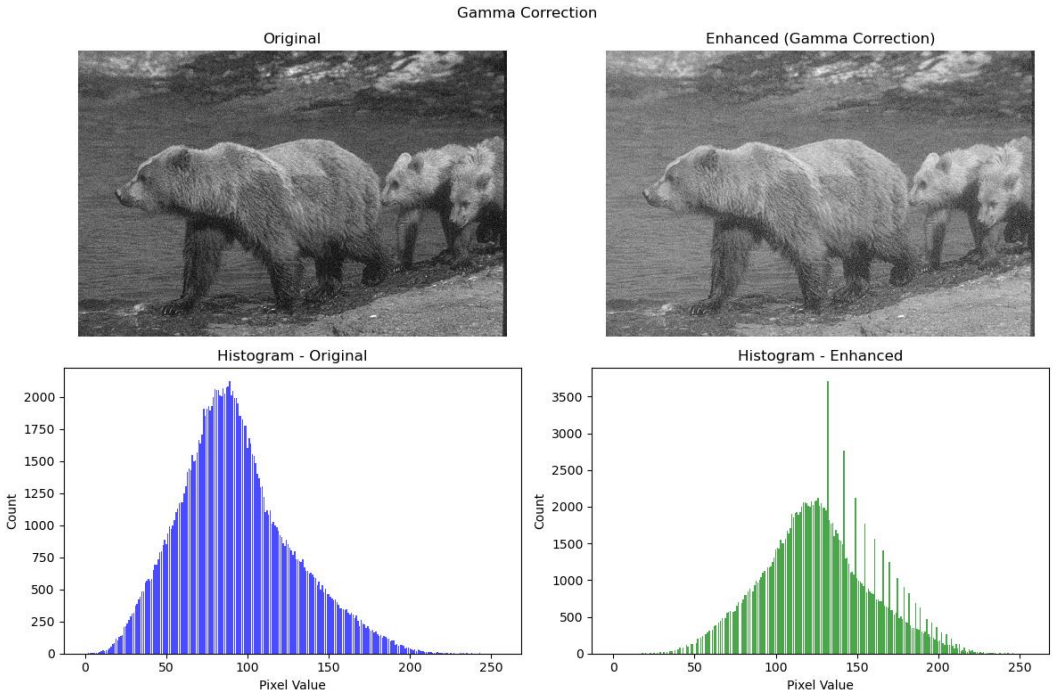
The third step is the application of non-maximum suppression, in which only pixels with the maximum gradient magnitude are retained at each edge point.Fourthly, double thresholds are employed to categorize each pixel as either a strong edge, a weak edge, or neither. The final step is known as hysteresis, whereby only those weak edge pixels connected to any strong edge pixels will be kept.

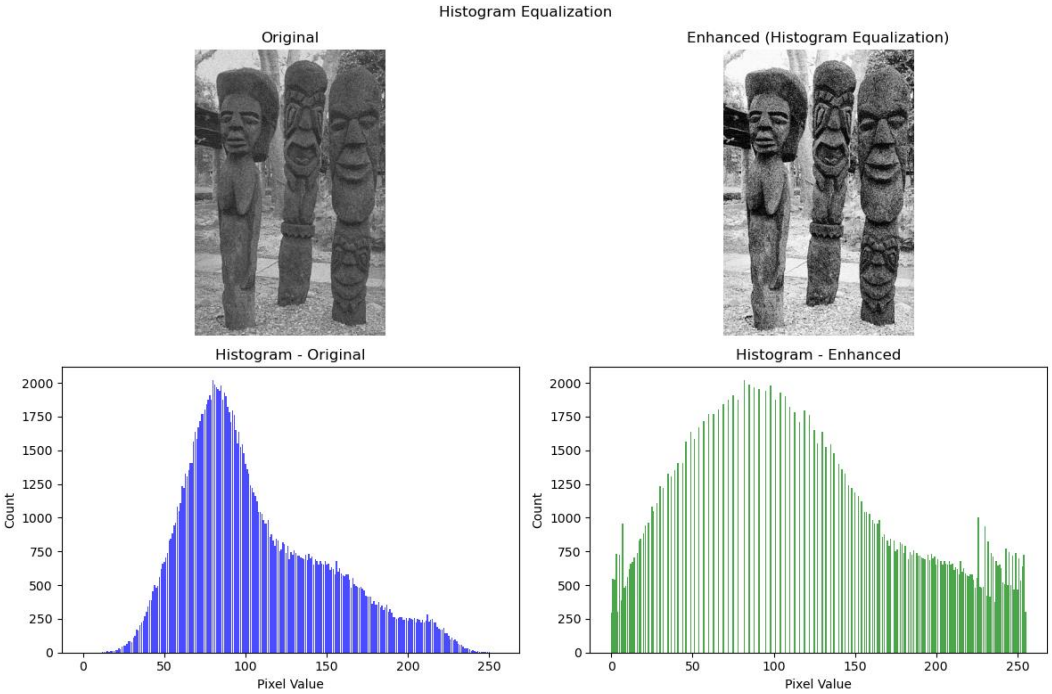
3.0 Results

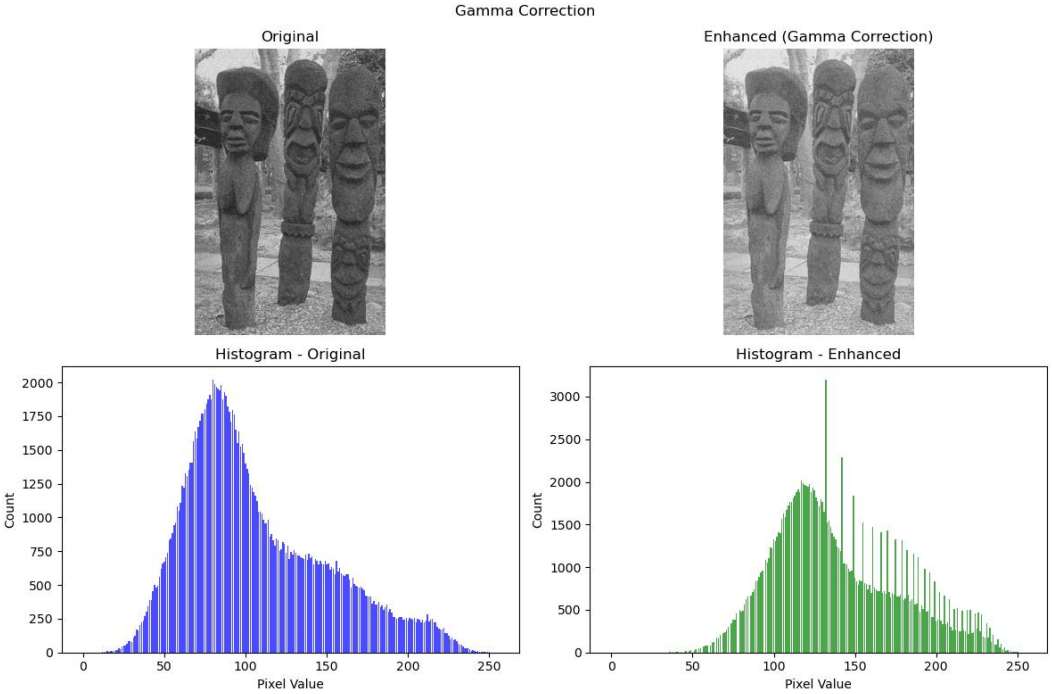
3.1 Image Enhancement Results

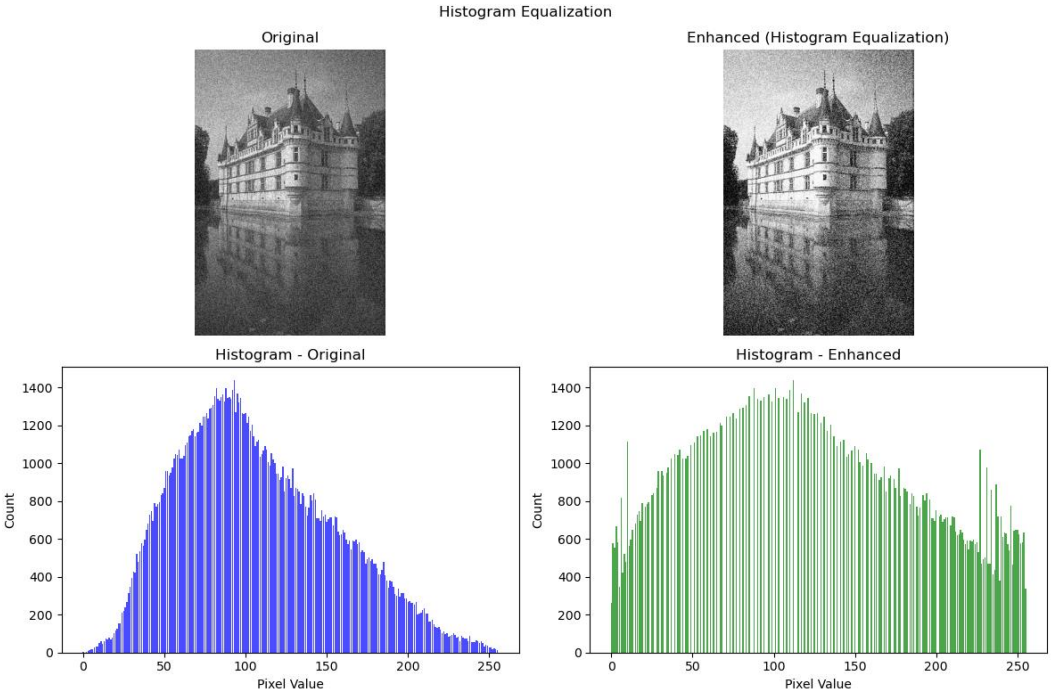
The enhancement techniques were applied to all images in the dataset. The figures below show four examples of the original images alongside their enhanced versions, along with the corresponding histograms before and after enhancement.

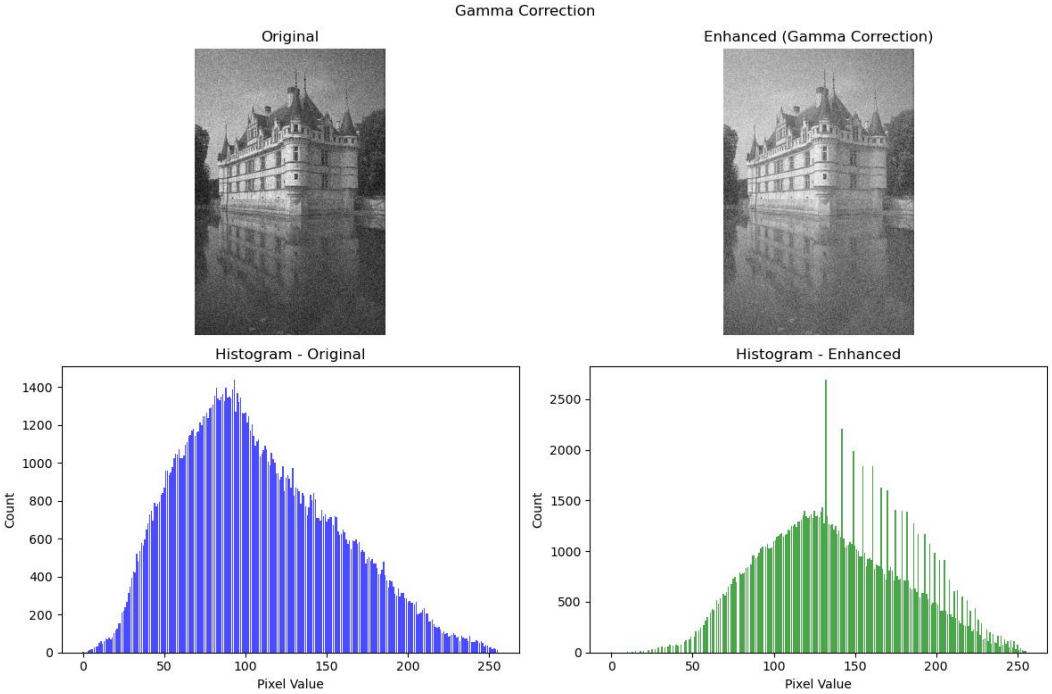





Histogram Equalization
Original
Enhanced (Histogram Equalization)
Histogram - Original
Histogram - Enhanced
Count
Pixel Value


Gamma Correction
Original
Enhanced (Gamma Correction)
Histogram - Original
Histogram - Enhanced
Count
Pixel Value


Histogram Equalization
Original
Enhanced (Histogram Equalization)
Histogram - Original
Histogram - Enhanced
Count
Pixel Value


Gamma Correction
Original
Enhanced (Gamma Correction)
Histogram - Original
Histogram - Enhanced
Count
Pixel Value

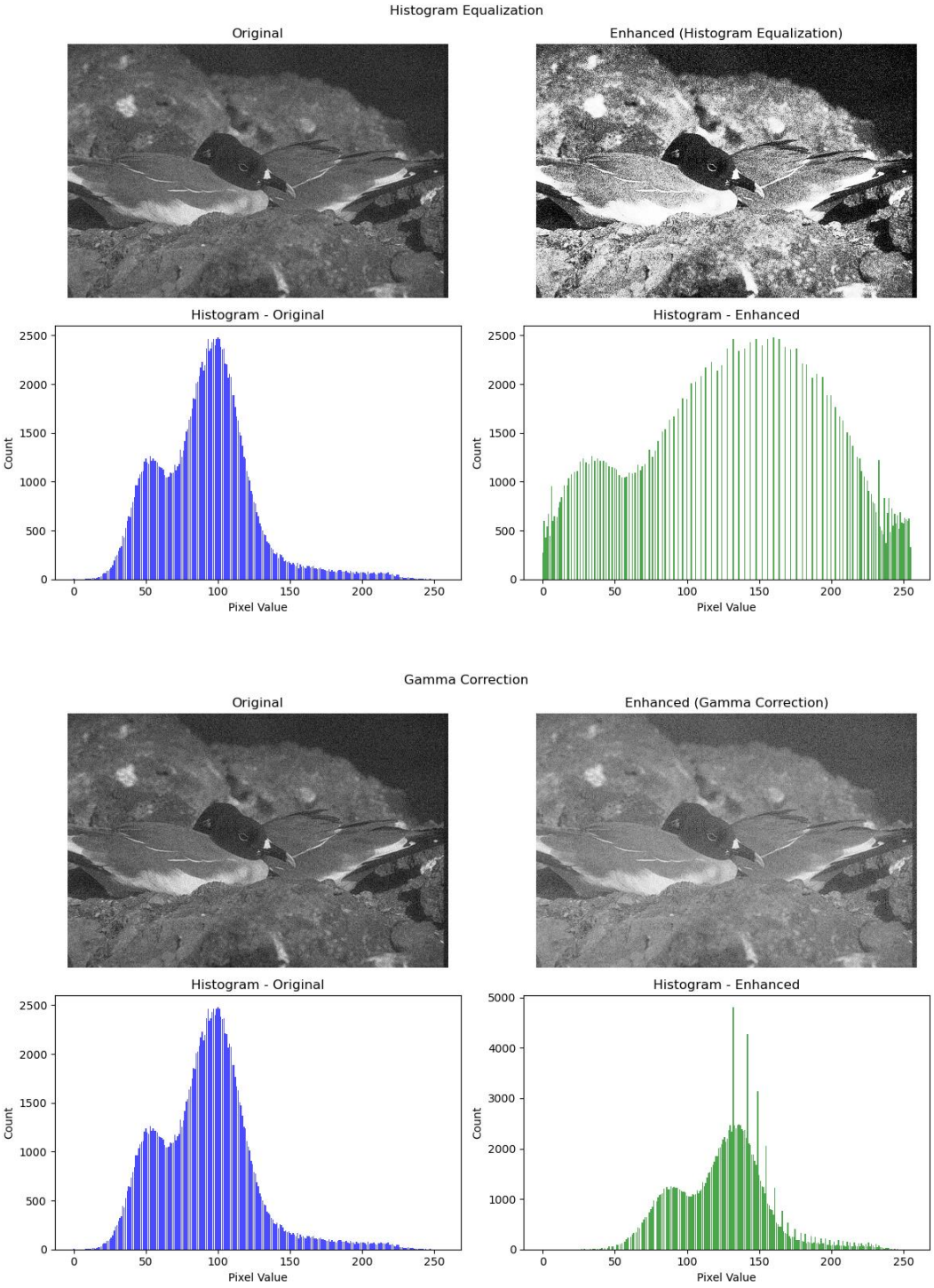


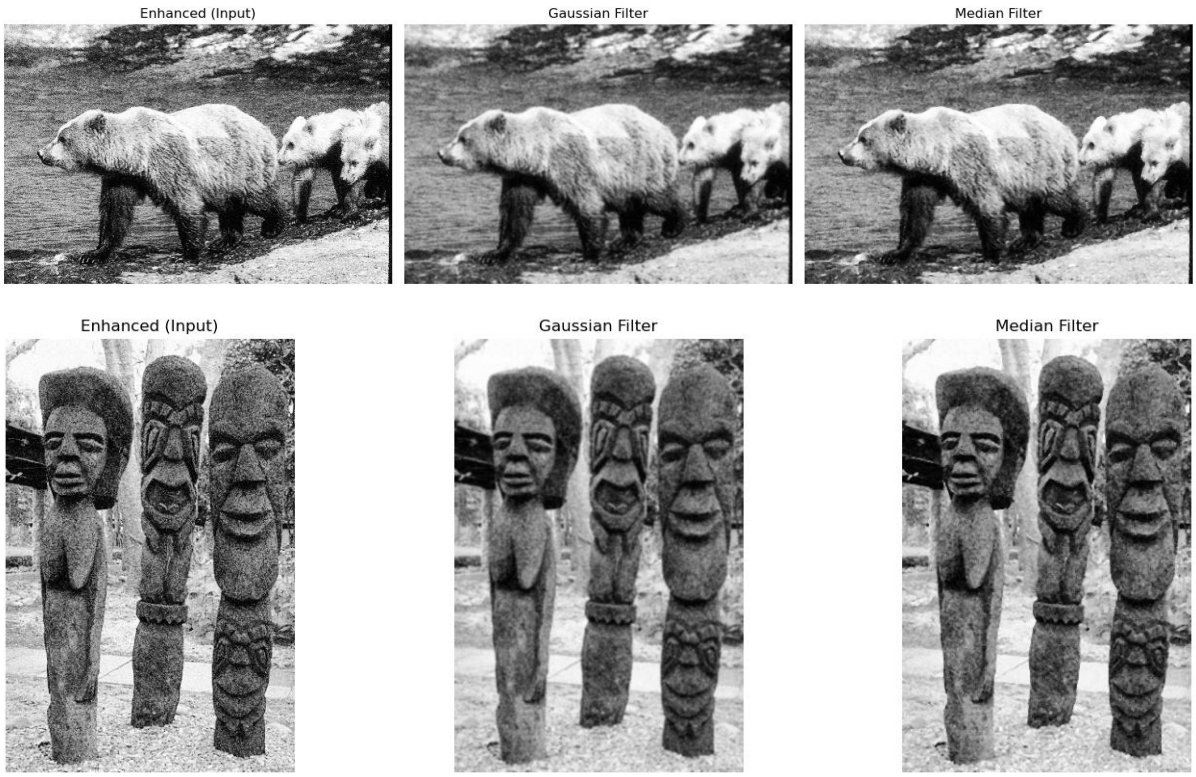
Figure 5: Four results of image enhancement using Histogram Equalization and Gamma Correction

For histogram equalization, the original images displayed histograms concentrated in a narrow range of pixel values, indicating low contrast. After equalization, the histograms became noticeably more spread across the full 0–255 range. Visually, the enhanced images showed stronger contrast with darker shadows and brighter highlights, making details more visible.

With gamma set to 1.5, dark regions in the images became visibly brighter. The overall brightness of the images improved without producing the unnatural patchy appearance that histogram equalization sometimes causes in already well-contrasted images.

3.2 Image Filtering Results

Both filtering techniques were applied to the enhanced images from Stage 1. The figures below show four examples comparing the enhanced input image with the Gaussian- and Median-filtered outputs.



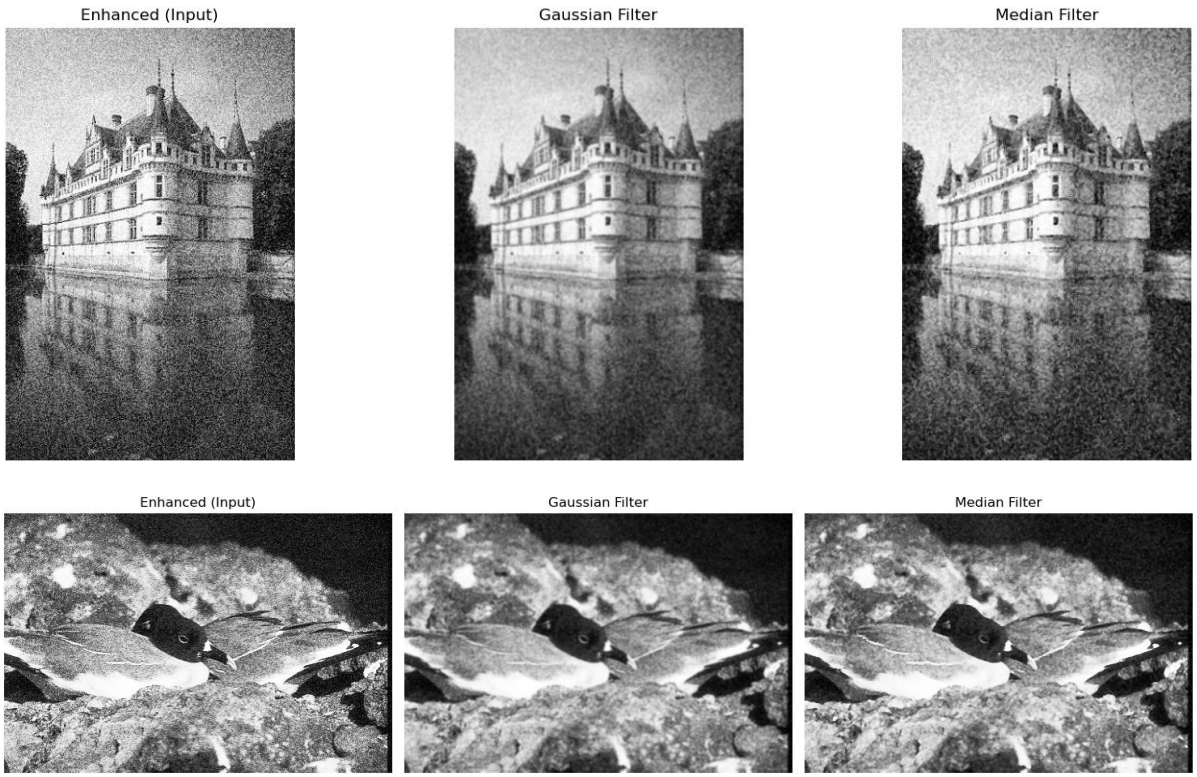
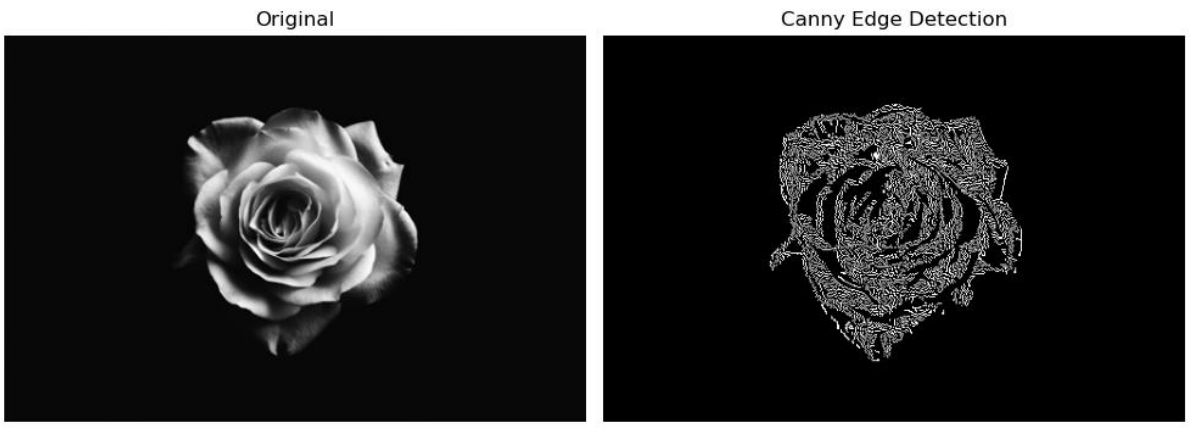


Figure 6: Four results of image filtering using the Gaussian and the Median filter

It is apparent that Gaussian filtering resulted in a smoother picture with decreased noise level. Nevertheless, the fine edges were somewhat blurred due to the averaging operation. Median filter successfully got rid of the noise specks, such as "salt & pepper" effect, leaving object borders sharper than the Gaussian filter.

3.3 Edge Detection Results

The Canny edge detector was applied to images from the edge detection folder in the dataset. Figures below show four examples of the original images alongside their detected edge maps.



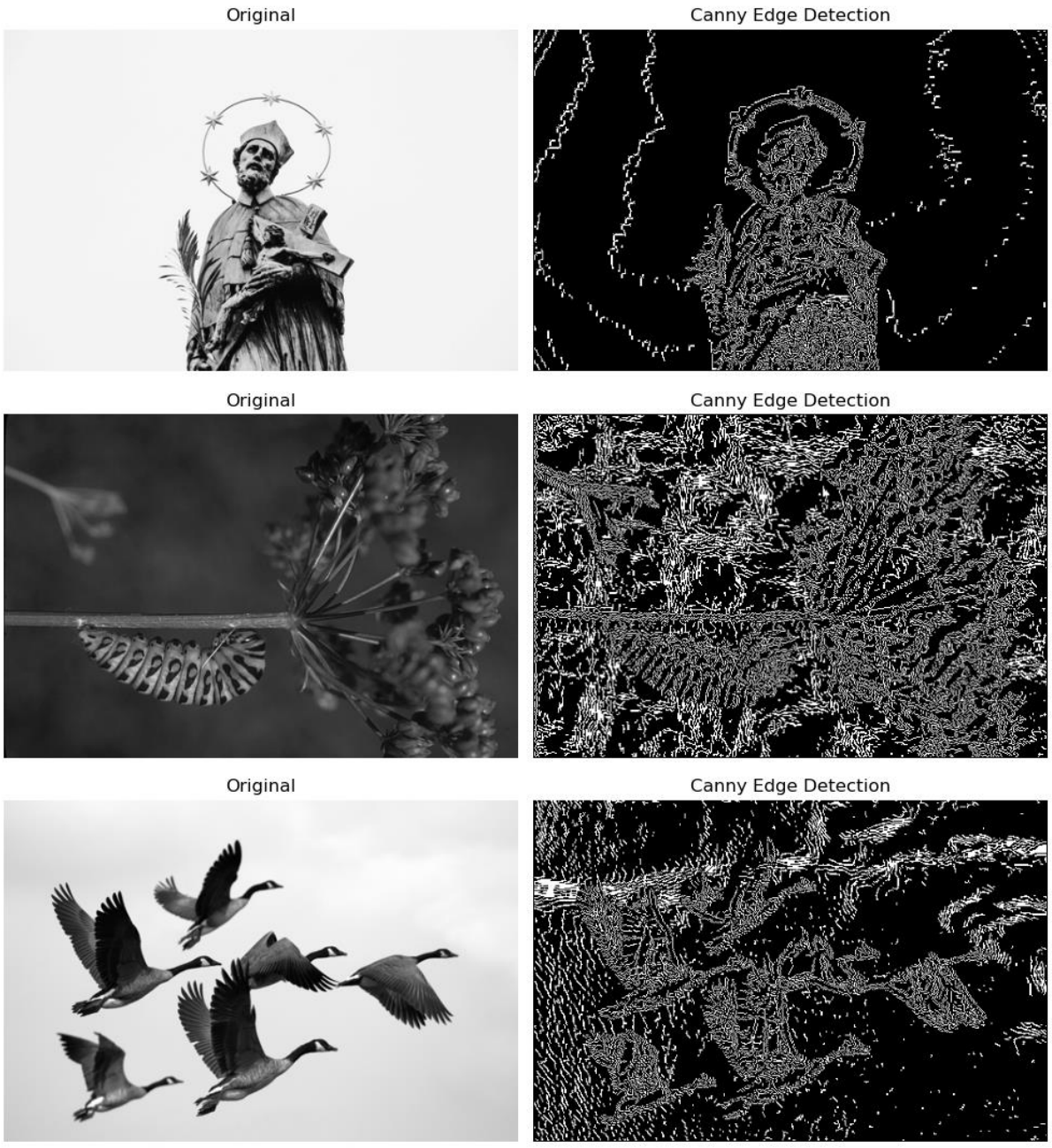


Figure 7: Four results of edge detection using the Canny Edge Detector

The obtained edge images contain thin white lines superimposed against a black background. The Canny edge detector was able to detect key edges, such as those between objects, as well as fine textures and structural boundaries. Smooth areas were not considered edges by the program, as expected.

4.0 Discussion

4.1 Enhancement Techniques Comparison

The histogram equalization method was quite effective in cases with very little contrast in the image, effectively concentrating all the pixels in a smaller portion of the intensity spectrum. This approach worked by spreading out the histogram throughout the entire spectrum, thus ensuring that the resulting image possessed good contrast. However, when used on an image with moderate contrast, the result would be images that appear overly enhanced.

On the other hand, gamma correction was quite effective in providing a brighter appearance to the image. Gamma correction works effectively on dark images since it enhances the darker pixels. Nevertheless, it does not correct contrast distribution problems, something that histogram equalization handles easily.

In conclusion, the two methods are quite complementary in nature, with one handling contrast issues and the other handling brightness issues.

4.2 Filtering Techniques Comparison

As far as removing noise from the image goes, the Gaussian filter functioned adequately and even excellently since it created an image devoid of any artifacts or other distortions. However, because it creates averages based on neighbouring pixels, it will always have some blurring effect, which might influence subsequent edge detection operations.

On the other hand, the median filter worked better in removing noise when compared to the Gaussian filter as it was able to detect the noise spike and remove it efficiently due to its nature as it uses a pixel value instead of averages.

Considering the given dataset, the median filter is deemed to be superior to the Gaussian one since it is better at balancing the two required qualities.

4.3 Edge Detection Analysis

Canny edge detector gave good results in the form of well-defined edges for all the test images. The use of initial Gaussian blurring was quite useful in detecting false edges due to noise, especially in images containing textured background areas. The non-maximal suppression technique helped thin the edges to give sharp one-pixel-wide edges.

The double thresholding technique helped in distinguishing between real edges and noise edges, while the hysteresis technique helped in connecting weak edge pixels with strong edge pixels to give complete edges. Image having high object boundary contrast and distinct boundaries produced best results through edge detection technique.

5.0 Conclusion

The following experiment implemented an image processing model made up of three different steps: image enhancement, image filtering, and edge detection. Histogram equalization was used to improve image contrast, while gamma correction was employed to improve image brightness. Gaussian filtering was carried out to eliminate noise from the images, while median filtering was done to eliminate salt and pepper noise from the images. The Canny edge detector was employed to detect object edges in five steps.

The following results show that each technique was effective and suitable under particular conditions of an image. Not all techniques performed better than the others on each type of image; therefore, it is necessary to determine the type of image before choosing a particular technique. Each step in the model has helped in achieving the desired output – an image with clear boundaries of the objects in the form of lines.

6.0 References

1. GeeksforGeeks. (2026, January 28). *Image Processing Techniques, Types & Applications*. GeeksforGeeks. https://www.geeksforgeeks.org/computer-vision/image-processing-techniques-types-applications/
2. GeeksforGeeks. (2025, July 23). *Histogram equalization in digital image processing*. GeeksforGeeks. https://www.geeksforgeeks.org/computer-graphics/histogram-equalization-in-digital-image-processing/
3. Galgani, F. (2024, October 10). *What Is Gamma Correction?* baeldung.com. Retrieved May 24, 2026, from https://www.baeldung.com/cs/gamma-correction-brightness
4. Cope, L. (2025, November 6). *How Gaussian Filters Remove Noise and Smooth Data - Engineer Fix*. Engineer Fix. https://engineerfix.com/how-gaussian-filters-remove-noise-and-smooth-data/
5. GeeksforGeeks. (2026b, May 6). *Implement Canny Edge Detector in Python using OpenCV*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/implement-canny-edge-detector-in-python-using-opencv/